

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Khoa Mạng máy tính và Truyền thông



UIT

TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

Môn học: Lập trình hệ thống

LAB 5: *Kỹ thuật dịch ngược*

Giảng viên hướng dẫn		Đỗ Thị Thu Hiền
Nhóm 11	24520557	Nguyễn Minh Hoàng
	24520602	Nguyễn Duy Hưng

Thành phố Hồ Chí Minh, ngày 27 tháng 11 năm 2025

CƠ CHẾ CHUNG CỦA VIỆC PHÂN TÍCH

Phân tích dựa trên các mã giả của các phase. Đối với những (tập) câu lệnh dẫn tới `explode_bomb()` – Bom nổ - Bị sai, ta xác định “điều ngược lại với điều kiện là đúng”.

PHASE 1

```
int __cdecl phase1(int a1)
{
    int result; // eax@3
    int v2[6]; // [sp+Ch] [bp-2Ch]@1
    int v3; // [sp+24h] [bp-14h]@3
    int v4; // [sp+28h] [bp-10h]@1
    int i; // [sp+2Ch] [bp-Ch]@5

    v4 = __isoc99_sscanf(a1, "%d %d %d %d %d %d", v2, &v2[1], &v2[2], &v2[3], &v2[4], &v2[5]);
    if ( v4 != 6 )
        explode_bomb();
    v3 = 1;
    result = v2[0];
    if ( v2[0] <= 1 )
        explode_bomb();
    for ( i = 1; i <= 5; ++i )
    {
        result = v2[i];
        if ( result != 2 * i + v2[i - 1] )
            explode_bomb();
    }
    return result;
}
```

1. Định dạng của input

- Định dạng và số lượng: Theo khai báo và ràng buộc v4, ta biết chuỗi 6 số nguyên.

```
int result; // eax@3
int v2[6]; // [sp+Ch] [bp-2Ch]@1
int v3; // [sp+24h] [bp-14h]@3
int v4; // [sp+28h] [bp-10h]@1
int i; // [sp+2Ch] [bp-Ch]@5

v4 = __isoc99_sscanf(a1, "%d %d %d %d %d %d", v2, &v2[1], &v2[2], &v2[3], &v2[4], &v2[5]);
if ( v4 != 6 )
    explode_bomb();
```

2. Điều kiện ràng buộc của input (giới hạn giá trị/ràng buộc giữa các phần tử)

a) Ràng buộc giá trị input

Chuỗi input chứa đúng 6 số nguyên.

```
if ( v4 != 6 )
    explode_bomb();
```

b) Điều kiện giá trị cho phần tử đầu tiên

v2[0] phải lớn hơn 1 (tức là $v2[0] \geq 2$).

```
result = v2[0];
if ( v2[0] <= 1 )
    explode_bomb();
```

c) Điều kiện ràng buộc theo vòng lặp (giữa các phần tử)

```
for ( i = 1; i <= 5; ++i )
{
    result = v2[i];
    if ( result != 2 * i + v2[i - 1] )
        explode_bomb();
}
```

Vòng lặp for chạy i từ 1 đến 5.

Điều kiện bên trong vòng lặp là: `if ( result != 2 * i + v2[i - 1] ) explode_bomb();`

Biến **result** trong vòng lặp này chính là `v2[i]`, vì nó được gán lại ở đầu mỗi lần lặp: `result = v2[i];`.

Do đó, điều kiện là: **`v2[i] = 2*i + v2[i - 1]`** với `i = 1, 2, 3, 4, 5`.

3. Kết luận về input của pha (liệt kê hoặc mô tả tất cả các input thỏa mãn)

Chương trình cho phép chọn số đầu tiên (`v2[0]`) ngẫu nhiên > 1 . Nên phụ thuộc vào số đầu tiên có thể chọn ra vô số chuỗi thỏa mãn với quy luật: **số sau = số trước + 2 * (thứ tự - 1)**.

Hoặc có thể biểu diễn lại là:

$v[1] = v[0] + 2$	$v[2] = v[0] + 6$	$v[3] = v[0] + 12$	$v[4] = v[0] + 20$	$v[5] = v[0] + 30$
-------------------	-------------------	--------------------	--------------------	--------------------

Ví dụ: 2 4 8 14 22 32

3 5 9 15 23 33

...

```
[*] Phase 1
- Hint: Numbers are always magical!
3 5 9 15 23 33
Good job! You've cleared the first phase!
```

4 - 5. Cách xác định đáp án submit – Kết quả

Đáp án điền là đáp án nhỏ nhất từ thành phần đầu tiên, mà theo ràng buộc, ta biết thành phần đầu tiên quyết định các thành phần sau nên ta chọn giá trị `v2[0]` nhỏ nhất có thể là 2.

Đáp án submit: 2 4 8 14 22 32

```
[*] Phase 1
- Hint: Numbers are always magical!
2 4 8 14 22 32
Good job! You've cleared the first phase!
```

PHASE 2:

Mã giả Phase 2:

```

1  int __cdecl phase2(int a1)
2  {
3      char *v1; // ST28_4@1
4      int result; // eax@2
5      char *s1; // [sp+Ch] [bp-1Ch]@1
6      char *s2; // [sp+10h] [bp-18h]@1
7
8      v1 = QUESTIONS[3];
9      s2 = ANSWERS[*( &QA_MAP + 3)];
10     s1 = (char *)transfer(a1);
11     if ( !*s2 || (result = is_equal(s1, s2)) == 0 )
12         explode_bomb();
13     return result;
14 }

```

1. Định dạng của input: Câu hỏi là quốc gia nào là thành phố sư tử ở phía nam nên input là chuỗi ký tự.

2. Điều kiện ràng buộc của input: Input phải là 1 chuỗi các ký tự và có độ dài bằng với đáp án.

- `v1 = QUESTIONS[3]`: Câu lệnh lấy ra câu hỏi thứ 4 trong mảng QUESTIONS ở đây là “Which country is the Lion city in South” gán vào `v1`.

<code>.data:0004C060 QUESTIONS</code>	<code>dd offset aWhatIsTheCapit ; DATA XREF: phase2+101r</code>
<code>.data:0004C060</code>	<code>; "What is the capital of Thailand?"</code>
<code>.data:0004C064</code>	<code>dd offset aWhatIsTheEngli ; "What is the English name of our course?"...</code>
<code>.data:0004C068</code>	<code>dd offset aWhichSeasonHas ; "Which season has cherry blossoms?"</code>
<code>.data:0004C06C</code>	<code>dd offset aWhichCountryIs ; "Which country is the Lion city in South"...</code>
<code>.data:0004C070</code>	<code>dd offset aIf2024IsTheYea ; "If 2024 is the year of Dragon, what ani"...</code>
<code>.data:0004C074</code>	<code>dd offset aWhichIsTheLarg ; "Which is the largest net in the world?"</code>
<code>.data:0004C078</code>	<code>dd offset aWhatIsTheLarge ; "What is the largest country in the worl"...</code>
<code>.data:0004C07C</code>	<code>dd offset aWhatWordIsSpel ; "What word is spelled incorrectly in eve"...</code>
<code>.data:0004C080</code>	<code>dd offset aWhatIsYourNati ; "What is your nationality?"</code>
<code>.data:0004C084</code>	<code>dd offset aWhatIsThePhone ; "What is the phone number of our univers"...</code>
<code>.data:0004C088</code>	<code>dd offset aCompleteTheDom ; "Complete the domain of our faculty: htt"...</code>
<code>.data:0004C08C</code>	<code>dd offset aWhatIsTheName0 ; "What is the name of the analyzed execut"...</code>
<code>.data:0004C090</code>	<code>dd offset aWhichCityHas37 ; "Which city has 3/7 of a chicken and 2/3"...</code>
<code>.data:0004C094</code>	<code>dd offset aInUitWhichMajo ; "In UIT, which major tops the list alpha"...</code>
<code>.data:0004C098</code>	<code>dd offset aThanksToMeYouC ; "Thanks to me, you can see straight thro"...</code>
<code>.data:0004C09C</code>	<code>dd offset aWhatIsTheLonge ; "What is the longest wall in the world?"...</code>
<code>.data:0004C0A0</code>	<code>dd offset aWhatIsTheEstab ; "What is the establishment date of UIT ("...</code>
<code>.data:0004C0A4</code>	<code>dd offset aWhatIsTheFullE ; "What is the full English name of our un"...</code>
<code>.data:0004C0A8</code>	<code>dd offset aIAmAnOddNumber ; "I am an odd number. Take away one lette"...</code>
<code>.data:0004C0AC</code>	<code>dd offset aWhichFastFoodB ; "Which fast food brand features a bee in"...</code>
<code>.data:0004C0B0</code>	<code>dd offset aWhichContinent ; "Which continent has the least populatio"...</code>
<code>.data:0004C0B4</code>	<code>dd offset aWhatAreEaxEbx ; "What are eax, ebx, eip, esi on a comput"...</code>
<code>.data:0004C0B8</code>	<code>dd offset aWhatIsTheSmall ; "What is the smallest planet in the Sola"...</code>
<code>.data:0004C0BC</code>	<code>dd offset aIsabellaSParen ; "Isabella's parents have four children. "...</code>
<code>.data:0004C0C0</code>	<code>dd offset aWhatIsTheDomai ; "What is the domain of UIT Students' ema"...</code>
<code>.data:0004C0C4</code>	<code>dd offset aWhatIsTheVietn ; "What is the Vietnamese name (without ac"...</code>
<code>.data:0004C0C8</code>	<code>dd offset aIfYouHaveItYou ; "If you have it, you want to share it. I"...</code>
<code>.data:0004C0CC</code>	<code>dd offset aWhatIsAlwaysCo ; "What is always coming but never arrives"...</code>
<code>.data:0004C0D0</code>	<code>dd offset aWhichEuropeanc ; "Which European country is shaped like a"...</code>

- `s2 = ANSWERS[*( &QA_MAP + 3)]`: Câu lệnh lấy giá trị thứ 4 của mảng QA_MAP ở đây là 3 rồi lấy tiếp ở mảng ANSWERS giá trị ở vị trí 3 ở đây là “Wmrketsvi” gán vào `s2`.

Lab 5 - Kỹ thuật Dịch ngược

```

.data:0004C160 ANSWERS dd offset aFerkoso ; DATA XREF: phase2+2A1r
.data:0004C160 ; "Ferkoso"
.data:0004C164 dd offset aGsqtyxiwWcwxiq ; "Gsqtyxiw Wcwxiq Tvskeqqmrk"
.data:0004C168 dd offset aWtmrk ; "Wtmrk"
.data:0004C16C dd offset aWmrketsui ; "Wmrketsui"
.data:0004C170 dd offset aWreoi ; "Wreoi"
.data:0004C174 dd offset aMrxivrix ; "Mrxivrix"
.data:0004C178 dd offset aUyvume ; "Uyvume"
.data:0004C17C dd offset aHrgsvuigxpc ; "Hrgsvuigxpc"
.data:0004C180 dd offset aZmixreqiwi ; "Zmixreqiwi"
.data:0004C184 dd offset a46271696446 ; "46271696446"
.data:0004C188 dd offset aRg_ymx_ihy_zr ; "rg.ymx.ihy.zr"
.data:0004C18C dd offset aRx643YmxFsQf ; "rx643-ymx-fsqf"
.data:0004C190 dd offset aGlmgeks ; "Glmgeks"
.data:0004C194 dd offset aEvxnjmgmepMrxi ; "Evxnjmgmep Mrxippmkirgi"
.data:0004C198 dd offset aAmrha ; "Amrha"
.data:0004C19C dd offset aXliKviexAeppSj ; "Xli Kviex Aepp Sj Glmre"
.data:0004C1A0 dd offset a42406440 ; "42/40/6440"
.data:0004C1A4 dd offset aVrmziuvumxcSjMr ; "Vrmziuvumxc Sj Mrjsvqexmsr Xiglrspskc"
.data:0004C1A8 dd offset aWizir ; "Wizir"
.data:0004C1AC dd offset aNsppmfii ; "Nsppmfii"
.data:0004C1B0 dd offset aErxevgxmge ; "Erxevgxmge"
.data:0004C1B4 dd offset aUikmxiow ; "Uikmxiow"
.data:0004C1B8 dd offset aQivgyuc ; "Qivgyuc"
.data:0004C1BC dd offset aMwefippe ; "Mwefippe"
.data:0004C1C0 dd offset aKq_ymx_ihy_zr ; "kq.ymx.ihy.zr"
.data:0004C1C4 dd offset aZerQmiyUysgXyK ; "Zer Qmiy Uysg Xy Kneq"
.data:0004C1C8 dd offset aWigvix ; "Wigvix"
.data:0004C1CC dd offset aXsqsvusa ; "Xsqsvusa"
.data:0004C1D0 dd offset aMxepc ; "Mxepc"

```

- `s1 = (char *)transfer(a1)`: Câu lệnh gọi hàm transfer để mã hóa input nhập vào. Sau đó kiểm tra xem s2 có phải chuỗi rỗng không hoặc s1 với s2 khác nhau hay input với câu trả lời khác nhau không (!*s2 || (result = is_equal(s1, s2)) == 0). Nếu có bomb sẽ nổ. Còn không thì input đã chính xác. Vậy nên muốn biết input là gì thì sẽ phải dùng hàm transfer để dịch ngược s2.
- **Hàm transfer**: Nhận vào một chuỗi và mã hóa trực tiếp từng ký tự trong chuỗi đó. Với các chữ cái (A–Z, a–z), nó dịch vòng 4 ký tự, giữ nguyên chữ hoa hay chữ thường. Với các chữ số (0–9), nó dịch vòng 4 trong phạm vi 0–9. Các ký tự khác không bị thay đổi. Kết quả cuối cùng là chuỗi input đã được mã hóa tại chỗ.

```

1 int __cdecl transfer(int a1)
2 {
3     char v2; // [sp+Ah] [bp-6h]@8
4     char v3; // [sp+Bh] [bp-5h]@2
5     int i; // [sp+Ch] [bp-4h]@1
6
7     // Duyệt qua từng ký tự của input
8     for ( i = 0; *(_BYTE *)(i + a1); ++i )
9     {
10        // Gán v3 bằng ký tự ở vị trí i
11        v3 = *(_BYTE *)(i + a1);
12        // Nếu không phải chữ A - Z hoặc không phải là chữ cái từ a - z
13        if ( (v3 <= 96 || v3 > 122) && (v3 <= 64 || v3 > 90) )
14        {
15            // Nếu là số
16            if ( v3 > 47 && v3 <= 57 )
17                // Mã hóa một chữ số bằng cách +4 trong phạm vi 0-9
18                v3 = (v3 - 48 + 4) % 10 + 48;
19        }
20        else
21        {
22            // Nếu v3 là chữ hoa
23            if ( v3 <= 96 || v3 > 122 )
24                v2 = 65;
25            else
26                v2 = 97;
27            // Mã hóa một chữ cái (hoa hoặc thường) bằng cách +4 trong phạm vi là chữ cái
28            v3 = (v3 - v2 + 4) % 26 + v2;
29        }
30        // Gán giá trị input tại vị trí i bằng v3 vừa tìm được
31        *(_BYTE *)(a1 + i) = v3;
32    }
33    return a1;
34 }

```

- Kết quả chạy hàm transfer:

Lab 5 - Kỹ thuật Dịch ngược

Ký tự mã hóa	W	m	r	k	e	t	s	v	i
Ký tự gốc	S	i	n	g	a	p	o	r	e

- Kết quả thực thi:

```
hung@LAPTOP-RI9PA5KK:/mnt/c/Users/ASUS/Downloads$ ./nt209-uit-bomb
Welcome to UIT's bomb lab.
You have to deactivate our bomb by solving 6 phases with the correct inputs consecutively, and otherwise the bomb will be blown up!

[*] Phase 1
- Hint: Numbers are always magical!
2 4 8 14 22 32
Good job! You've cleared the first phase!

[*] Phase 2
- Hint: You must answer your secret question!
Singapore
Two phases have been solved. Keep going!
```

```

int __cdecl phase3(int a1)
{
    int result; // eax@21
    unsigned __int8 v2; // [sp+Fh] [bp-19h]@1
    int v3; // [sp+10h] [bp-18h]@1
    int v4; // [sp+14h] [bp-14h]@1
    int v5; // [sp+18h] [bp-10h]@1
    char v6; // [sp+1Fh] [bp-9h]@4

    v5 = 0;
    v5 = __isoc99_sscanf(a1, "%d %c %d", &v4, &v2, &v3);
    if ( v5 <= 2 )
        explode_bomb();
    switch ( v4 )
    {
        case 0:
            v6 = 113;
            if ( v3 != 251 )
                explode_bomb();
            return result;
        case 1:
            v6 = 103;
            if ( v3 != 561 )
                explode_bomb();
            return result;
        case 2:
            v6 = 98;
            if ( v3 != 695 )
                explode_bomb();
            return result;
        case 3:
            v6 = 102;
            if ( v3 != 324 )
                explode_bomb();
            return result;
        case 4:
            v6 = 102;
            if ( v3 != 916 )
                explode_bomb();
            return result;
        case 5:
            v6 = 118;
            if ( v3 != 440 )
                explode_bomb();
            return result;
        case 6:
            v6 = 112;
            if ( v3 != 201 )
                explode_bomb();
            return result;
        case 7:
            v6 = 99;
            if ( v3 != 938 )
                explode_bomb();
            return result;
        default:
            v6 = 97;
            explode_bomb();
            return result;
    }
    result = v2;
    if ( v6 != v2 )
        explode_bomb();
    return result;
}

```

1. Định dạng của input

Định dạng: Ba phần tử lần lượt: số nguyên – ký tự - số nguyên.

```

int result; // eax@21
unsigned __int8 v2; // [sp+Fh] [bp-19h]@1
int v3; // [sp+10h] [bp-18h]@1
int v4; // [sp+14h] [bp-14h]@1
int v5; // [sp+18h] [bp-10h]@1
char v6; // [sp+1Fh] [bp-9h]@4

v5 = 0;
v5 = __isoc99_sscanf(a1, "%d %c %d", &v4, &v2, &v3);
if ( v5 <= 2 )
    explode_bomb();

```

2. Điều kiện ràng buộc của input (giới hạn giá trị/ràng buộc giữa các phần tử)

a) Điều kiện số lượng đọc thành công

```

v5 = 0;
v5 = __isoc99_sscanf(a1, "%d %c %d", &v4, &v2, &v3);
if ( v5 <= 2 )
    explode_bomb();

```

Hàm phải đọc được hơn 2 phần tử.

b) Ràng buộc giá trị v4 – v6 – v3 – v2

* Thông qua vòng lặp switch:

- v4 có giá trị từ 0-7.
- Giá trị v6 phụ thuộc vào giá trị v4 (case 0: v6 = 113; case 1: v6 = 103;...)
- Giá trị v3 phụ thuộc vào giá trị v4 (case 0: v3 = 251; case 1: v3 = 561;...)

* v2 phụ thuộc vào v6 thông qua lệnh if cuối:

```
result = v2;
if ( v6 != v2 )
    explode_bomb();
```

Giá trị v2 phụ thuộc vào giá trị v6.

Kết luận ràng buộc: Từ v4 có thể suy ra giá trị của v6 và v3, và gián tiếp tìm ra giá trị của v2 và truy về result.

3. Kết luận về input của pha (liệt kê hoặc mô tả tất cả các input thỏa mãn)

Chương trình có 7 trường hợp (dựa theo 7 giá trị của v4). Vì v2 là dạng ký tự nên biến đổi theo ASCII.

Các đáp án khả thi:

0 – q – 251 1 – g – 561 2 – b – 695 3 – f – 324 4 – f – 916
5 – v – 440 6 – p – 201 7 – c – 938

```
switch ( v4 )
{
    case 0:
        v6 = 113;
        if ( v3 != 251 )
            explode_bomb();
        return result;
    case 1:
        v6 = 103;
        if ( v3 != 561 )
            explode_bomb();
        return result;
    case 2:
        v6 = 98;
        if ( v3 != 695 )
            explode_bomb();
        return result;
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
0 q 251
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
1 g 561
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
2 b 695
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
3 f 324
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
4 f 916
You've beaten another phase, that's great. What about the fourth one?
```



```
[*] Phase 3
- Hint: Many cases make everything so confusing.
5 v 440
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
6 p 201
You've beaten another phase, that's great. What about the fourth one?
```

```
[*] Phase 3
- Hint: Many cases make everything so confusing.
7 c 938
You've beaten another phase, that's great. What about the fourth one?
```

4 – 5. Cách xác định đáp án đã submit – Kết quả

Đáp án nhỏ nhất tính từ thành phần đầu tiên (v4) mà v4 quyết định v2, v3 nên ta chỉ cần chọn v4 nhỏ nhất = 0 => 0 – q – 251.

PHASE 4:

1. Định dạng input: 2 số nguyên vì hàm `__isoc99_sscanf(a1, "%d %d", &v2, &v3)` nhận 2 số nguyên lần lượt là v2 và v3.

2. Điều kiện ràng buộc của input: Đầu vào phải đủ 2 số. Số thứ 2 nhập vào phải có giá trị từ 2 đến 5 vì có câu lệnh `if (v6 != 2 || v3 <= 1 || v3 > 5)` kiểm tra v3 nằm trong khoảng 2 đến 5, nếu vượt thì bomb nổ.

3. Mã giả phase 4:

```
1  int __cdecl phase4(int a1)
2  {
3      int result; // eax@5
4      int v2; // [sp+Ch] [bp-1Ch]@1
5      int v3; // [sp+10h] [bp-18h]@1
6      int v4; // [sp+14h] [bp-14h]@5
7      int v5; // [sp+18h] [bp-10h]@5
8      int v6; // [sp+1Ch] [bp-Ch]@1
9
10     v6 = __isoc99_sscanf(a1, "%d %d", &v2, &v3);
11     if ( v6 != 2 || v3 <= 1 || v3 > 5 )
12         explode_bomb();
13     v5 = 6;
14     v4 = func4(6, v3);
15     result = v2;
16     if ( v4 != v2 )
17         explode_bomb();
18     return result;
19 }
```

- `v4 = func4(6, v3)`: Gán v4 bằng kết quả của hàm `func4(6, v3)`. Ta lần lượt cho các giá trị của v3 và lấy ra các kết quả của các giá trị v4:
- Mã giả hàm `func`:

```
1  int __cdecl func4(int a1, int a2)
2  {
3      int result; // eax@2
4      int v3; // ebx@5
5
6      if ( a1 > 0 )
7      {
8          if ( a1 == 1 )
9          {
10             result = a2;
11         }
12         else
13         {
14             v3 = func4(a1 - 1, a2) + a2;
15             result = v3 + func4(a1 - 2, a2);
16         }
17     }
18     else
19     {
20         result = 0;
21     }
22     return result;
23 }
```

Lab 5 - Kỹ thuật Dịch ngược

- Chuyển đổi sang mã c++ để dùng vs code để chạy hàm func4 với các giá trị của v3 cho dễ dàng hơn:

```
1 #include <iostream>
2 using namespace std;
3
4 int __cdecl func4(int a1, int a2)
5 {
6     int result;
7     int v3;
8
9     if ( a1 > 0 )
10    {
11        if ( a1 == 1 )
12        {
13            result = a2;
14        }
15        else
16        {
17            v3 = func4(a1 - 1, a2) + a2;
18            result = v3 + func4(a1 - 2, a2);
19        }
20    }
21    else
22    {
23        result = 0;
24    }
25    return result;
26 }
27
28 int main() {
29     for (int i = 2; i <= 5; i++) {
30         int ans = func4(6, i);
31         cout << ans << " : " << i << "\n";
32     }
33 }
```

- Kết quả sau khi chạy hàm nhận được các giá trị tương ứng của v4 lần lượt là 40, 60, 80, 100
- Hình ảnh kết quả thực thi:

```
40 : 2
60 : 3
80 : 4
100 : 5
```

- Giá trị của v4 chính là giá trị của v2 vì có câu lệnh kiểm tra if (v4 != v2) nếu v4 khác v2 thì bomb nổ.
- Vậy các kết quả của phase 4 là các cặp v2 v3 vừa tìm được:
+ 40 2 + 60 3 + 80 4 + 100 5
=> Kết quả bé nhất là: 40 2
- Kết quả thực thi:

```
[*] Phase 4
- Hint: Let's dig in to recursive function :)
40 2
Congrats! You've found another correct input. Moving ahead!

[*] Phase 4
- Hint: Let's dig in to recursive function :)
60 3
Congrats! You've found another correct input. Moving ahead!

[*] Phase 4
- Hint: Let's dig in to recursive function :)
80 4
Congrats! You've found another correct input. Moving ahead!

[*] Phase 4
- Hint: Let's dig in to recursive function :)
100 5
Congrats! You've found another correct input. Moving ahead!
```

```

size_t __cdecl phase5(char *s)
{
    size_t result; // eax@1
    int u2; // [sp+8h] [bp-10h]@3
    signed int i; // [sp+Ch] [bp-Ch]@3

    result = strlen(s);
    if ( result != 6 )
        explode_bomb();
    u2 = 0;
    for ( i = 0; i <= 5; ++i )
    {
        result = array_3853[s[i] & 0xF];
        u2 += result;
    }
    if ( u2 != 66 )
        explode_bomb();
    return result;
}

```

```

.data:0804C240 ; int array_3853[]
.data:0804C240 array_3853      dd 2
.data:0804C244                db 0Ah
.data:0804C245                db 0
.data:0804C246                db 0
.data:0804C247                db 0
.data:0804C248                db 6
.data:0804C249                db 0
.data:0804C24A                db 0
.data:0804C24B                db 0
.data:0804C24C                db 1
.data:0804C24D                db 0
.data:0804C24E                db 0
.data:0804C24F                db 0
.data:0804C250                db 0Ch
.data:0804C251                db 0
.data:0804C252                db 0
.data:0804C253                db 0
.data:0804C254                db 10h
.data:0804C255                db 0
.data:0804C256                db 0
.data:0804C257                db 0
.data:0804C258                db 9
.data:0804C259                db 0
.data:0804C25A                db 0
.data:0804C25B                db 0
.data:0804C25C                db 3
.data:0804C25D                db 0
.data:0804C25E                db 0
.data:0804C25F                db 0

```

```

.data:0804C260                db 4
.data:0804C261                db 0
.data:0804C262                db 0
.data:0804C263                db 0
.data:0804C264                db 7
.data:0804C265                db 0
.data:0804C266                db 0
.data:0804C267                db 0
.data:0804C268                db 0Eh
.data:0804C269                db 0
.data:0804C26A                db 0
.data:0804C26B                db 0
.data:0804C26C                db 5
.data:0804C26D                db 0
.data:0804C26E                db 0
.data:0804C26F                db 0
.data:0804C270                db 0Bh
.data:0804C271                db 0
.data:0804C272                db 0
.data:0804C273                db 0
.data:0804C274                db 8
.data:0804C275                db 0
.data:0804C276                db 0
.data:0804C277                db 0
.data:0804C278                db 0Fh
.data:0804C279                db 0
.data:0804C27A                db 0
.data:0804C27B                db 0
.data:0804C27C                db 0Dh
.data:0804C27D                db 0
.data:0804C27E                db 0
.data:0804C27F                db 0
.data:0804C27F _data        ends

```

1. Định dạng của input

Định dạng: dạng chuỗi ký tự có độ dài bằng 6.

2. Điều kiện ràng buộc của input (giới hạn giá trị/ràng buộc giữa các phần tử)

a) Giới hạn giá trị

Hàm phải có độ dài bằng 6.

b) Ràng buộc giá trị phần tử v2

- Tổng giá trị của v2 phải bằng 66.

- Giá trị của v2 được cộng dồn trong vòng lặp, câu lệnh gán cho result có ý nghĩa là: lấy 4 bit thấp nhất của mã ASCII của ký tự s[i]. Mảng array_3853 (hình trên).

```

u2 = 0;
for ( i = 0; i <= 5; ++i )
{
    result = array_3853[s[i] & 0xF];
    u2 += result;
}

```

3. Kết luận về input của pha (liệt kê hoặc mô tả tất cả các input thỏa mãn)

Vì chúng ta chỉ quan tâm tới 4 bit nhỏ nhất nên có thể có nhiều ký tự ASCII có cùng 4 bit cuối.

Index	Giá trị	Ký tự ASCII (")
0	2	0, p
1	10 (0A)	1, a, q
2	6	2, b, r
3	1	3, c, s
4	12 (0C)	4, d, t
5	16 (10)	5, e, u
6	9	6, f, v
7	3	7, g, w
8	4	8, h, x
9	7	9, i, y
10	14 (0E)	., j, z
11	5	;, k
12	11 (0B)	<, l
13	8	=, m
14	15 (0F)	>, n
15	13 (0D)	?, o

Input có thể là 1 chuỗi 6 ký tự theo cột 3, sao cho tổng giá trị của 6 ký tự (chiều theo cột 2) là 66.

Ví dụ:

Giá trị: $2 + 2 + 16 + 16 + 16 + 14 = 66 \Rightarrow 00555:$

Giá trị: $11 + 11 + 11 + 11 + 11 + 11 = 66 \Rightarrow \text{llllll}$

```
[*] Phase 5
-Hint: No hint is also a hint :)
llllll
Awesome! Only one phase left!
```

4 - 5. Cách xác định đáp án đã submit – Kết quả

Ưu tiên 1: Các ký tự ASCII có cùng Index 4 bit cuối nhưng mã ASCII nhỏ nhất.

Index	Giá trị	Ký tự ASCII nhỏ nhất (")
0	2	0
1	10 (0A)	1
2	6	2
3	1	3
4	12 (0C)	4
5	16 (10)	5
6	9	6
7	3	7
8	4	8
9	7	9
10	14 (0E)	:
11	5	;
12	11 (0B)	<
13	8	=
14	15 (0F)	>
15	13 (0D)	?

Ưu tiên 2: Đáp án nhỏ nhất từ thành phần đầu tiên.

‘0’ là ký tự có mã ASCII nhỏ nhất $\Rightarrow$ Ưu tiên chọn được nhiều ký tự ‘0’ nhất có thể.

Gọi chuỗi ký tự là xxxxxx, tổng giá trị 66.

- Ký tự ‘0’ có giá trị 2.

- Ký tự ‘5’ có giá trị cao nhất = 16.

Gọi số ký tự ‘0’ trong chuỗi là k:

$$2.k + 16.(6 - k) \leq 66 \Rightarrow k \leq 2,14 \Rightarrow \text{Max}(k) = 2.$$

Khi k = 2: Gọi n là số ký tự ‘5’ trong chuỗi:

$$2.2 + 16.n \leq 66 \Leftrightarrow n \leq 3,875 \Rightarrow \text{Max}(n) = 3.$$

Lab 5 - Kỹ thuật Dịch ngược

Gọi m là giá trị của ký tự còn lại:

$$2.2 + 16.3 + 1.m = 66 \Rightarrow m = 14.$$

Vậy các ký tự của chuỗi bao gồm: 0 0 5 5 5 :

Khi sắp xếp, chúng ta dựa theo vị trí ASCII chứ không dựa theo giá trị riêng. 0 (0) – 5 (53) - : (58) thì 00555: là đáp án nhỏ nhất.

```
[*] Phase 5
-Hint: No hint is also a hint :)
00555:
Awesome! Only one phase left!
```

PHASE 6:

- 1. Định dạng input:** 6 số nguyên vì hàm `__isoc99_sscanf(a1, "%d %d %d %d %d %d", v3, v4, v5, v6, v7, v8)` nhận đầu vào là 6 số nguyên.
- 2. Điều kiện ràng buộc của input:** input phải đủ 6 số.
- 3. Mã giả phase 6:**

```

1  int __cdecl phase6(int a1)
2  {
3      int result; // eax@23
4      int v2[6]; // [sp+Ch] [bp-4Ch]@18
5      int v3[6]; // [sp+24h] [bp-34h]@1
6      int v4[6]; // [sp+28h] [bp-30h]@1
7      int v5[6]; // [sp+2Ch] [bp-2Ch]@1
8      int v6[6]; // [sp+30h] [bp-28h]@1
9      int v7[6]; // [sp+34h] [bp-24h]@1
10     int v8[6]; // [sp+38h] [bp-20h]@1
11     int v9; // [sp+3Ch] [bp-1Ch]@1
12     int v10; // [sp+40h] [bp-18h]@1
13     int j; // [sp+44h] [bp-14h]@7
14     int i; // [sp+48h] [bp-10h]@3
15     int v13; // [sp+4Ch] [bp-Ch]@15
16
17     v10 = (int)&node1;
18     v9 = __isoc99_sscanf(a1, "%d %d %d %d %d %d", v3, v4, v5, v6, v7, v8);
19     if ( v9 != 6 )
20         explode_bomb();
21     for ( i = 0; i <= 5; ++i )
22     {
23         if ( v3[i] <= 0 || v3[i] > 6 )
24             explode_bomb();
25         for ( j = i + 1; j <= 5; ++j )
26         {
27             if ( v3[i] == v3[j] )
28                 explode_bomb();
29         }
30     }
31     for ( i = 0; i <= 5; ++i )
32     {
33         v13 = v10;
34         for ( j = 1; v3[i] > j; ++j )
35             v13 = *(_DWORD *)(v13 + 8);
36         v2[i] = v13;
37     }
38     v10 = v2[0];
39     v13 = v2[0];
40     for ( i = 1; i <= 5; ++i )
41     {
42         *(_DWORD *)(v13 + 8) = v2[i];
43         v13 = *(_DWORD *)(v13 + 8);
44     }
45     *(_DWORD *)(v13 + 8) = 0;
46     result = v10;
47     v13 = v10;
48     for ( i = 0; i <= 4; ++i )
49     {
50         if ( *(_DWORD *)v13 < **(_DWORD **)(v13 + 8) )
51             explode_bomb();
52         result = *(_DWORD *)(v13 + 8);
53         v13 = *(_DWORD *)(v13 + 8);
54     }
55     return result;
56 }

```

- `v10 = (int)&node1`: Gán `v10` bằng địa chỉ của node đầu tiên.
- Vòng lặp thứ 1: Kiểm tra các số nhập vào phải có giá trị từ 1 đến 6 (`v3[i] <= 0 || v3[i] > 6`) và không có số nào được trùng lặp (vòng lặp con bên trong). Mặc dù nhập `v3, v4, v5, v6, v7, v8` nhưng chỉ duyệt bằng `v3` vì địa chỉ các mảng liền kề nhau.

```

1  for ( i = 0; i <= 5; ++i )
2  {
3      if ( v3[i] <= 0 || v3[i] > 6 )
4          explode_bomb();
5      for ( j = i + 1; j <= 5; ++j )
6      {
7          if ( v3[i] == v3[j] )
8              explode_bomb();
9      }
10 }

```

- Vòng lặp thứ 2: Duyệt qua tất cả các node trong danh sách liên kết đơn để tìm địa chỉ node tương ứng với số đã nhập. v13 lưu địa chỉ của node1. Duyệt qua danh sách node để lấy địa chỉ của node tại vị trí số đã nhập. Gán địa chỉ tìm được vào vị trí tương ứng trong mảng v2.

```

1  for ( i = 0; i <= 5; ++i )
2  {
3      v13 = v10;
4      for ( j = 1; v3[i] > j; ++j )
5          v13 = *(_DWORD *)(v13 + 8);
6      v2[i] = v13;
7  }

```

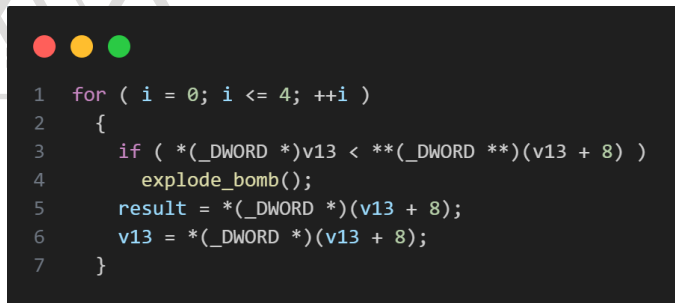
- v10 = v2[0]: Lưu địa chỉ của head để trả về.
- v13 = v2[0]: Lưu địa chỉ của head để duyệt ở vòng lặp sau.
- Vòng lặp thứ 3: Thay đổi giá trị của các node theo giá trị đã tìm được ở v2

```

1  for ( i = 1; i <= 5; ++i )
2  {
3      *(_DWORD *)(v13 + 8) = v2[i];
4      v13 = *(_DWORD *)(v13 + 8);
5  }

```

- result = v10: Gán result bằng địa chỉ đầu tiên của danh sách liên kết
- v13 = v10: Gán v13 bằng địa chỉ đầu tiên của danh sách liên kết
- Vòng lặp thứ 4: Duyệt qua danh sách (đã được bạn sắp xếp lại ở vòng 3) và kiểm tra giá trị bên trong. Giá trị của Node trước phải **LỚN HƠN** giá trị Node sau. Để qua màn, ta phải sắp xếp các Node theo thứ tự **GIÁ TRỊ GIẢM DẦN** (từ to đến nhỏ).



⇒ Kết luận thuật toán của cả bài là sắp xếp các node trong danh sách liên kết đơn sao cho có giá trị giảm dần.

- Giá trị các node:

+ Node 6: 96 01 00 00 => Hệ 16: 0x00000196 => Hệ 10: 406

<code>.data:0804C1EC</code>	<code>public node6</code>
<code>.data:0804C1EC</code>	<code>db 96h ; 0</code>
<code>.data:0804C1ED</code>	<code>db 1</code>
<code>.data:0804C1EE</code>	<code>db 0</code>
<code>.data:0804C1EF</code>	<code>db 0</code>

+ Node 5: 82 02 00 00 => Hệ 16: 0x00000282 => Hệ 10: 642

<code>.data:0804C1F8</code>	<code>public node5</code>
<code>.data:0804C1F8</code>	<code>db 82h ; 2</code>
<code>.data:0804C1F9</code>	<code>db 2</code>
<code>.data:0804C1FA</code>	<code>db 0</code>
<code>.data:0804C1FB</code>	<code>db 0</code>

+ Node 4: 45 02 00 00 => Hệ 16: 0x00000245 => Hệ 10: 581

<code>.data:0804C204</code>	<code>public node4</code>
<code>.data:0804C204</code>	<code>db 45h ; E</code>
<code>.data:0804C205</code>	<code>db 2</code>
<code>.data:0804C206</code>	<code>db 0</code>
<code>.data:0804C207</code>	<code>db 0</code>

+ Node 3: B7 00 00 00 => Hệ 16: 0x000000B7 => Hệ 10: 183

<code>.data:0804C210</code>	<code>public node3</code>
<code>.data:0804C210</code>	<code>db 0B7h ; +</code>
<code>.data:0804C211</code>	<code>db 0</code>
<code>.data:0804C212</code>	<code>db 0</code>
<code>.data:0804C213</code>	<code>db 0</code>

+ Node 2: 1A 02 00 00 => Hệ 16: 0x0000021A => Hệ 10: 538

<code>.data:0804C21C</code>	<code>public node2</code>
<code>.data:0804C21C</code>	<code>db 1Ah</code>
<code>.data:0804C21D</code>	<code>db 2</code>
<code>.data:0804C21E</code>	<code>db 0</code>
<code>.data:0804C21F</code>	<code>db 0</code>

+ Node 1: AC 01 00 00 => Hệ 16: 0x000001AC => Hệ 10: 428

<code>.data:0804C228</code>	<code>public node1</code>
<code>.data:0804C228</code>	<code>db 0ACH ; ¼</code>
<code>.data:0804C229</code>	<code>db 1</code>
<code>.data:0804C22A</code>	<code>db 0</code>
<code>.data:0804C22B</code>	<code>db 0</code>

⇒ Kết quả cuối cùng sau khi sắp xếp là: 5 4 2 1 6 3

- Hình ảnh thực thi:

[*] Phase 6

-Hint: Can you master single linked list?

5 4 2 1 6 3

Amazing bomb solvers, the bomb has been deactivated. Enjoy your day :))

hung@LAPTOP-RI9PA5KK:/mnt/c/Users/ASUS/Downloads\$ |